

Manual Técnico

Diagrama Entidad Relacion

Para el esquema de base de datos, se manejan 3 tablas relacionadas de manera tal que puedan dar una lógica funcional a la aplicación en relación al consumo de datos.



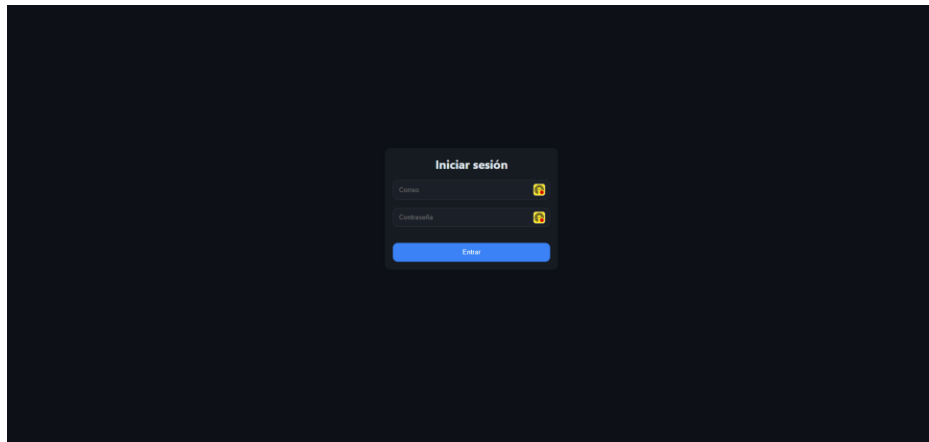
Diagrama de Arquitectura

Se maneja una estructura básica tomando en consideración un backend nodejs, front ReactJS, un middleware para consumo de BD por Windows authentication por entorno local, y dockerizado para su globalización.

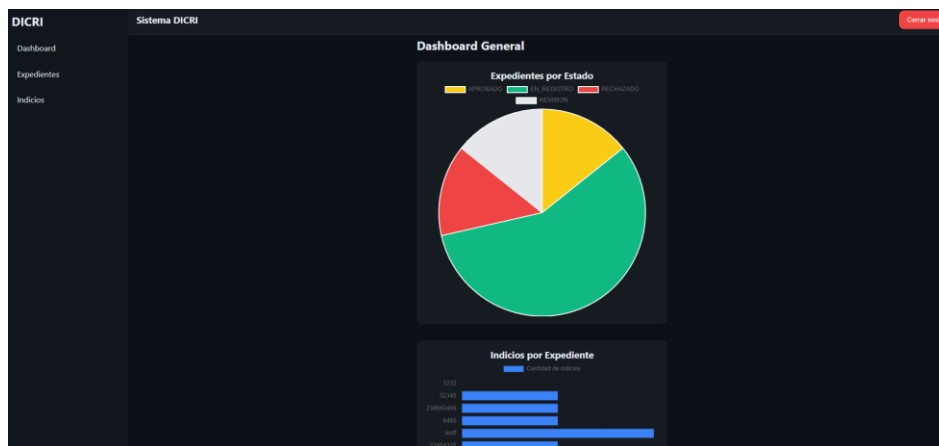
Resumen Técnico

La solución busca dar un sistema de ingreso a usuarios autorizados a nivel interno por BD (sin opción a registro de usuario voluntario por ser plataforma administrativa). También pretende manejar el ingreso de expedientes, ingreso de incidentes relacionados por expediente, gestión y actualización de estados por expediente y seguimiento por autorización o rechazo por usuarios coordinadores. Cuenta también con una sección básica para gráficos representativos en relación a los datos existentes y relevantes del aplicativo.

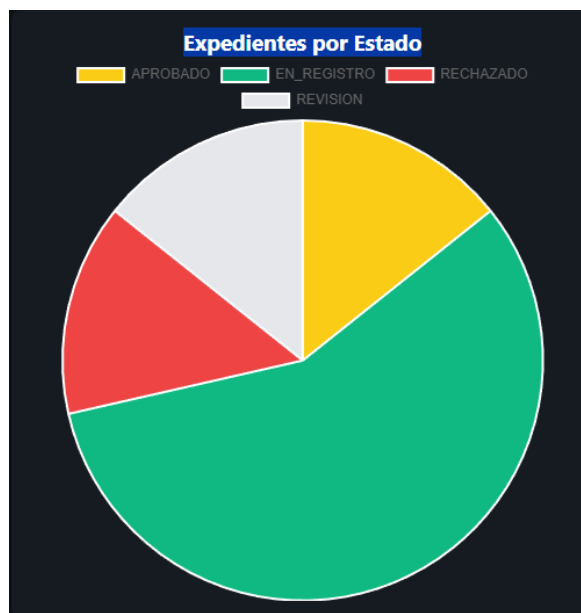
LoginPage



DashboardPage o Página de gráficos



Expedientes por estado: Esta muestra la totalidad porcentual de expedientes existentes y su magnitud de estados.



Indicios por expediente: Este muestra la cantidad de indicios existentes por expediente en el sistema.



Expedientes por usuario: Esto muestra la cantidad de expedientes registrados por usuario en el sistema.



Expedientes Aprobados/Rechazados: Este muestra la cantidad de expedientes rechazados y aprobados en totalidad del sistema



Expedientes por mes: Este muestra la cantidad de expedientes ingresados en un mes determinado a máximo 4 meses de antigüedad al mes presente.



ExpedientesPage (Tecnico)

En esta pagina se ingresan los expedientes, se listan los expedientes existentes con paginación y se muestra por expediente el resumen y capacidad de ingreso de incidentes a un expediente o paso a revisión, todo esto bajo la perspectiva de un usuario no coordinador.

DICRI

Dashboard

Expedientes

Indicios

Sistema DICRI

Cerrar sesión

Nuevo Expediente

Crear

Listado de Expedientes

Código / Descripción	Estado
EXP-002 Segundo expediente de prueba	en_registro
6465 asdasdfasdf	en_registro
234563456 asdasdfutgud	en_registro
1232 asdasdfasdfasdf	en_registro

1

EXP-002

Segundo expediente de prueba

Indicios

No hay indicios

Nuevo Indicio

Agregar indicio

Pasar a revisión

ExpedientesPage (Coordinador)

Para la perspectiva de un usuario con rol Coordinador se puede agregar expedientes, ver la lista de expedientes en revisión, aprobados y rechazados, en el caso de los rechazados se muestra el motivo del rechazo, y en los estado en revisión se pueden seleccionar para su posterior autorización o rechazo.

Código / Descripción	Estado
23454325 asdfsdfasdf	Aprobado
asdf asdfsdfasdf	Rechazado
52345 asdfsdfasdf	En revisión

Consumo de BD

Se establece por msnodesqlv8 la configuración a BD por medio de Windows Authentication y configuración .ENV para información sensible.

```
const sql = require('mssql/msnodesqlv8');    "msnodesqlv": Unknown word.
const dotenv = require('dotenv');
dotenv.config();

const config = {
  database: process.env.DB_NAME,
  server: process.env.DB_HOST,
  driver: "msnodesqlv8",    "msnodesqlv": Unknown word.
  options: {
    trustedConnection: true, // Windows Authentication
  }
};

let poolPromise = null;

async function getPool() {
  if (!poolPromise) {
    try {
      poolPromise = sql.connect(config);
      console.log("Conexión exitosa a SQL Server con Windows Authentication");    "C
    } catch (err) {
      console.error("Error conectando a SQL:", err);    "conectando": Unknown word.
      throw err;
    }
  }
  return poolPromise;
}
```

Ruta de repositorio

https://github.com/Mariocrv208/DICRI_MarioRodriguez.git

Implementacion de Token JWT (Middleware)

Se estructura la validación por JWT token para el acceso al sistema, configurando información sensible por configuraciones .ENV.

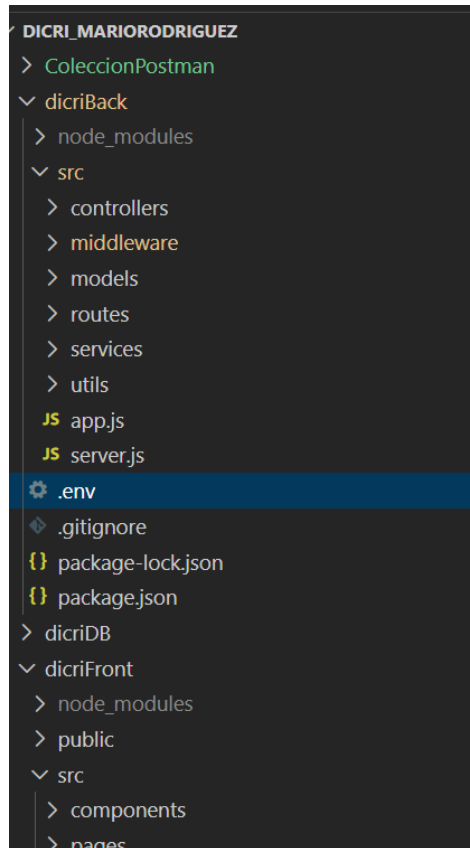
```
const jwt = require('jsonwebtoken');
const dotenv = require('dotenv');
dotenv.config();

exports.verifyToken = (req, res, next) => {
  const header = req.headers['authorization'];
  if (!header) return res.status(401).json({ message: 'No token provided' });
  const token = header.split(' ')[1];
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = payload;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Token inválido' });    "inválido":
  }
};

exports.requireRole = (role) => (req, res, next) => {
  if (!req.user) return res.status(401).json({ message: 'No autorizado' });
  if (req.user.rol !== role && req.user.rol !== 'admin') {
    return res.status(403).json({ message: 'Acceso denegado' });    "Acceso":
  }
  next();
};
```

Esquema de proyecto (Estructura de carpetas)

Se estructura de manera tal que evite el llamado “Codigo espaguetti”, asegurando las buenas practicas de codificación.



Gitflow

Se maneja división en la estructura de ramas para asegurar el correcto flujo de git y correcto control de versiones.



Docker